Builder vs Factory Pattern — Team Briefing & Q&A

Folder: `src/main/java/com/example/knowledge_sharing/exercise` | Gaming Example (Warrior, Mage, Archer)

1. Opening Pitch (Say This First)

- **Builder (`exercise/builder`) = Custom Hero Maker (1 Single Object):** Caller feeds 5 stats step-by-step (`weapon` , `armor` , `specialAbility` , `level` , `comboSkills`) into **1 single Hero object**.
- **Abstract Factory (`exercise/factory`) = Starter Kit Box (Multiple Matching Objects):** Caller calls `spawnCharacter()` once; factory creates **3 separate matching item objects** (`Weapon` + `Armor` + `SpecialAbility`).
- **YAGNI ("You Aren't Gonna Need It"):** Kept each folder to **4 files** by using lambdas in Factory and combining Builder + Product in Builder until extra features are needed.

2. Workflow & Dataflow Diagram

A. Builder Workflow (Outside-In → 1 Hero Object)

Caller: main() — `new WarriorBuilder()`

↓

Parent: CharacterBaseBuilder
1. `.weapon("Greatsword")` | 2. `.armor("Plate Armor")` | 3. `.specialAbility("Berserk Slash")`

↓ *(Covariant override returns WarriorBuilder)*

Child: WarriorBuilder
4. `.level(10)` | 5. `.comboSkills("Shield Bash + Whirlwind")`

↓ `.build()`

Output: 1 Single Complex Hero Object (`WarriorBuilder` with 5 fields)

B. Factory Workflow (Inside-Out → 3 Item Objects)

Caller: main() — `new MageCreator()`

↓

Parent Template: CharacterCreator — calls `spawnCharacter()`

↓ *(Calls 3 factory methods)*

Object 1	Object 2	Object 3
`Weapon` "Arcane Staff"	`Armor` "Silk Robe"	`Skill` "Fireball"

↓ *(Joins .name() of all 3)*

Output: 3 Matching Objects Combined (`"Arcane Staff + Silk Robe + Fireball"`)

3. Comparison Table: Pros, Cons & YAGNI

Topic	Builder (`exercise/builder`)	Abstract Factory (`exercise/factory`)
What It Creates	**1 Single Hero Object** packed with 5 fields	**3 Separate Matching Objects** (`Weapon` , `Armor` , `SpecialAbility`)
Data Direction	**Outside-In:** Caller passes all 5 values in `main()`	**Inside-Out:** Values pre-saved inside Creator class
YAGNI Shortcut	`WarriorBuilder` is both Builder and final Hero (saved 4 files)	Used lambdas `() -> "Greatsword"` instead of 9 item files
Pros	• Full customization per hero (`level(10)` , `comboSkills`) • Clear step-by-step chain instead of 5-arg constructor • Can skip optional fields	• **Zero wrong-gear bug:** Mage always gets Mage gear • 1-click spawn (`spawnCharacter()`) • Easy to add new class family (`PaladinCreator`)
Cons	• **Cursed loadout risk:** Can give Mage `"Plate Armor"` • Verbose: 6 lines of code per hero • Hero still mutable after `.build()`	• **Clones only:** Cannot set custom `level` or weapon • Adding 4th item (`createBoots()`) breaks all subclasses

4. Live Q&A Session & When to Use

Q1: Both create a Warrior in main(). What is the core difference? — **Builder** makes **1 single Hero object** with custom stats (`level` , `comboSkills`). **Factory** makes **3 separate matching item objects** (`Weapon` , `Armor` , `SpecialAbility`) so gear sets never mix.

Q2: What happens if a developer writes new MageBuilder().weapon("Greatsword").armor("Plate Armor")? — Builder allows it! Builder gives freedom, not family validation. Use **Factory** when gear must strictly match the class.

Q3: Why didn't we create Greatsword.java, PlateArmor.java, SilkRobe.java in exercise/factory? — **YAGNI**. Items only have `name()` right now. Writing `() -> "Greatsword"` creates a real `Weapon` object in 1 line instead of 9 extra files. Extract separate classes only when weapons need damage or durability.

Q4: Why does WarriorBuilder.build() return WarriorBuilder instead of a separate Warrior class? — Also **YAGNI**. Separate `Warrior` , `Mage` , `Archer` classes would duplicate all 5 fields. Split into an immutable class only when sharing heroes across threads.

Q5: What if our game needs BOTH safe default gear AND custom level(10)? — Combine them: let `WarriorCreator` return a pre-filled `WarriorBuilder` with default Warrior gear, then chain `.level(10).comboSkills(...).build()` .

Overall Conclusion (When to Use):
• **Pick Factory** for **default bots / starter kits** where **multiple objects must match as a family**.
• **Pick Builder** for **custom players / bosses** where **1 complex object** needs custom runtime stats.
• **Keep YAGNI**: Use lambdas and single classes today; only add extra files when real game rules require them.